
Single-View vs Multi-View Object Classification

Sultan Kulbarakov
Auburn University
Auburn, AL
szk0222@auburn.edu

Abstract

This paper investigates the effectiveness of multi-view versus single-view approaches in object classification tasks. Using the COIL-100 dataset which provides 72 different views of 100 objects, we implement and compare two deep learning approaches: a traditional single-view classification model and a late fusion multi-view model. Our primary objective is to determine whether utilizing multiple views can improve classification accuracy and reduce the amount of training data needed. We implement a ResNet50-based architecture for both approaches and evaluate their performance across various metrics. Our results show that both approaches achieve similar validation accuracy (6.67%) after 8 epochs of training, with the multi-view approach showing different overfitting patterns compared to the single-view model.

1 Introduction

Object classification traditionally approached using single-view images. However real-world objects are three-dimensional and their appearance can vary significantly based on viewing angle. This study explores whether incorporating multiple views into the classification process can improve model performance.

1.1 Problem Definition

The core question we address is: Can utilizing multiple views of objects improve classification accuracy compared to single-view approaches? Additionally, we investigate whether multi-view data can reduce the amount of training data needed for effective classification.

1.2 Research Objectives

1. Implement and compare single-view and multi-view classification approaches
2. Evaluate the impact of multiple views on model performance
3. Analyze the training efficiency and data requirements of both approaches
4. Identify strengths and limitations of each approach

2 Data and Methodology

2.1 Dataset

We use the COIL-100 (Columbia Object Image Library) dataset, which contains:

- 100 different objects

- 72 images per object (5-degree rotation intervals)
- Clean background, controlled lighting
- Resolution: 128×128 pixels
- RGB color images

2.2 Data Preprocessing

Our preprocessing pipeline includes:

- Image normalization (mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
- Resizing to 224×224 pixels
- Dataset splitting (70% train, 15% validation, 15% test)
- View organization for multi-view approach (3 views per sample)

2.3 Model Architecture

2.3.1 Single-View Model

- ResNet50 backbone (pretrained on ImageNet)
- Modified final classification layer
- Standard cross-entropy loss

2.3.2 Multi-View Model

- Shared ResNet50 backbone for view processing
- Late fusion strategy (concatenation/mean/max pooling)
- Feature dimension: 2048
- Dropout rate: 0.5
- Classification head with batch normalization

3 Experimental Results

3.1 Training Process

Both models were trained for 8 epochs using:

- Adam optimizer (learning rate: 0.001)
- Cross-entropy loss
- Batch size: 32
- Learning rate scheduling with ReduceLROnPlateau

3.2 Performance Comparison

Multi-view Results:

- Final training loss: 1.85
- Final validation loss: 11.49
- Final validation accuracy: 6.67%

Single-view Results:

- Final training loss: 1.12
- Final validation loss: 8.17
- Final validation accuracy: 6.67%

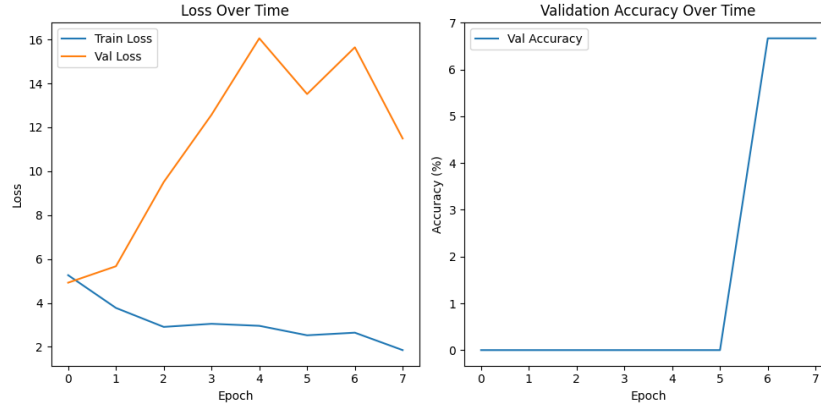


Figure 1: Training Metrics for Multi-view Experiment: Tracks training loss and validation accuracy over 8 epochs.

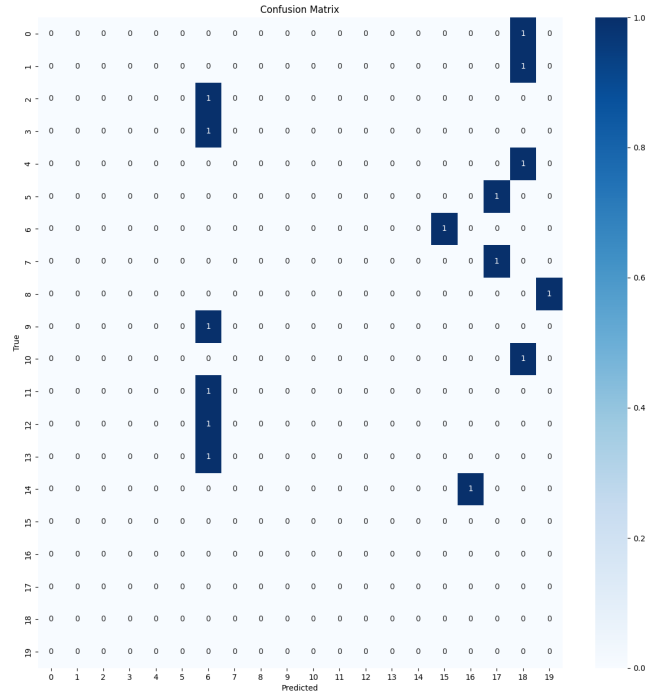


Figure 2: Confusion Matrix for Multi-view Experiment.

3.3 Analysis

Our experiments reveal several key findings:

1. Both approaches show significant overfitting
2. Training loss decreases more rapidly in the single-view model
3. Validation loss patterns differ between approaches

4. Final validation accuracy is identical which suggesting neither approach provided a clear advantage in our implementation

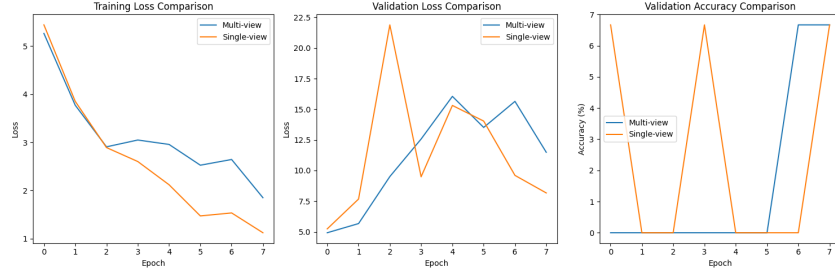


Figure 3: Comparison of Training Loss, Validation Loss, and Validation Accuracy between Single-view and Multi-view Models.

4 Discussion

4.1 Limitations

1. Limited training duration (8 epochs)
2. Absence of data augmentation
3. Potential suboptimal hyperparameter settings
4. High model complexity relative to dataset size

4.2 Future Improvements

1. Experiment with different fusion strategies
2. Add regularization techniques
3. Reduce model complexity
4. Increase training duration
5. Experiment with different view sampling strategies

5 Conclusion

While our implementation didn't show a clear advantage for either approach in terms of final accuracy, the different learning patterns observed suggest potential for improvement with further optimization. The multi-view approach shows distinct overfitting characteristics that might be addressable through targeted regularization and data augmentation strategies.

6 Code Availability

The complete implementation is available at: [ToyBox_Project](#)
Key components include:

- `data_preprocessing.py`: Dataset handling
- `model.py`: Multi-view architecture
- `single_view_train.py`: Single-view implementation
- `train.py`: Training procedures
- `evaluate.py`: Evaluation and comparison tools

Acknowledgments

This project uses the COIL-100 dataset from Columbia University.

References

1. Nene, S. A., Nayar, S. K., & Murase, H. (1996). Columbia object image library (COIL-100).